

Metamorphic Code Generation from LLVM Bytecode

Teja Tamboli* Thomas H. Austin[†] Mark Stamp[‡]

Abstract

Metamorphic software changes its internal structure across generations with its functionality remaining unchanged. Metamorphism has been employed by malware writers as a means of evading signature detection and other advanced detection strategies. However, code morphing also has potential security benefits, since it can serve to increase the “genetic diversity” of software.

We have created a metamorphic code generator within the LLVM compiler framework. LLVM is a three-phase compiler that supports multiple source languages and target architectures. It uses a common intermediate representation (IR) bytecode in its optimizer. Consequently, any supported high-level programming language is transformed to this IR bytecode as part of the LLVM compilation process. Our metamorphic generator functions at the IR bytecode level, which provides many advantages over morphing at the assembly or source code level. The morphing techniques that we employ include dead code insertion and transposition, where the dead code is actually executed within the morphed code, making its detection and removal more challenging. We have verified the effectiveness of our code morphing using hidden Markov model analysis.

1 Introduction

Software is said to be metamorphic if multiple copies are structurally different, but functionally equivalent. Examples of metamorphic malware generators can be found in [7, 8, 17, 32].

To date, metamorphic code generation has primarily been used by malware writers, since well-designed metamorphic code can evade signature-based detection and other advanced detection strategies [17, 32, 38]. However, metamorphism also has the potential to provide security benefits by increasing the “genetic diversity” of software, thereby making several types of attacks more difficult and by limiting the damage of successful attacks [11, 34].

*Department of Computer Science, San Jose State University

[†]Department of Computer Science, San Jose State University

[‡]Department of Computer Science, San Jose State University: stamp@cs.sjsu.edu

Many metamorphic malware generators are readily available at [25]. Some notable examples include

- G2 (Second Generation virus generator) [25]
- MPCGEN (Mass Code Generator) [37]
- NGVCK (Next Generation Virus Creation Kit) [37]
- VCL32 (Virus Creation Lab for Win32) [1]
- MetaPHOR [7]

In addition, research morphing engines are presented in [17] and [32]. All of these metamorphic generators work at the assembly language level. Code morphing of high-level source code is far simpler, but much less effective, since such morphing does not provide sufficient control over the resulting executable file.

In this research, we have implemented and analyzed a metamorphic code generator built on the LLVM compiler framework [14, 21]. LLVM is a three-phase compiler that supports multiple source languages and multiple target architectures. In the optimization process, code is converted to intermediate representation (IR) bytecode. Our code morphing tool functions at this IR bytecode level, which simplifies many types of morphing (analogous to working at the source code level), but also provides the necessary fine-grained control over the resulting executable (analogous to working at the assembly code level).

Related research involving LLVM IR bytecode manipulation includes a malware encryption technique implemented as optimizer passes [30]. In [24], a “shadow attack” is developed using LLVM. This attack hides system call behavior for the purpose of making behavior-based detection of malware more difficult.

We evaluate our morphing technique using a hidden Markov model (HMM) analysis similar to that in [13], which, in turn, is derived from the HMM-based malware detector analyzed in [38]. This HMM technique has been used as a baseline for comparing other proposed metamorphic detection strategies [3, 17, 28, 29, 32, 36]. This body of work provides a firm basis for analyzing the effectiveness of our morphing approach.

The paper is organized as follows. In Section 2.1, we provide relevant background information. Section 3 covers the design and implementation of our metamorphic code generator. Experimental results are analyzed in Section 4 while Section 5 contains our conclusion and suggestions for future work.

2 Background

In this section we briefly discuss the following topics: malware, metamorphic techniques, the LLVM compiler infrastructure, and hidden Markov models. Each of these topics is relevant to the work presented in Sections 3 and 4.

2.1 Malware

Malware is software that is designed to perform malicious activity [26]. To date, most development and research into metamorphic code has involved malware. Therefore, we present a brief introduction to metamorphic malware before turning our attention to the general case of metamorphic code generation.

2.1.1 Malware Evolution

In this section, we briefly consider the evolution that has led to the development of metamorphic malware. Below, we use the term virus generically to refer to malware.

Since signature detection is the most common anti-virus (AV) technique, virus writers have developed a variety of strategies for evading such detection. Perhaps the simplest method to hide a virus body from static signature detection is to encrypt or pack the executable. For encrypted malware, simple schemes are generally used, such as an XOR of each byte with a fixed value [2], which is equivalent to a simple substitution cipher. The malware writer’s goal is to obfuscate the code, so simple encryption schemes suffice. However, decryption code must be included, and that code is not encrypted, which opens the door to signature scanning [9].

To make detection more difficult, malware writers have developed so-called polymorphic code, where the virus body is encrypted (or packed) and the decryption code is morphed between generations. Consequently, there is no fixed signature for the decryptor code, making signature detection far more difficult [2]. However, polymorphic code is subject to detection via emulator—the code will eventually decrypt itself at which point it is subject to standard signature detection [9].

To avoid signature detection by emulation, malware writers have developed “body polymorphic” or metamorphic malware. Metamorphic code changes its internal structure at each generation, without altering its function. Well-designed metamorphic malware will exhibit no common signature and hence there is no need for encryption [9].

2.2 Metamorphic Techniques

In this section, we discuss several elementary metamorphic techniques. To date, most hacker-produced metamorphic malware has used only relatively simple morphing strategies. We also mention a relatively sophisticated morphing technique based on formal grammars.

2.2.1 Register Swap

Register swapping is one of the simplest code morphing techniques. For example, `PUSH ECX` can be replaced by `PUSH EAX`, provided the `EAX` register is not in use. Note that register swapping does not affect opcode sequences. Furthermore, a wildcard string can be used to overcome register swapping [6].

2.2.2 Transposition

Subroutine swapping is another elementary morphing technique. If a program has n subroutines, then $n!$ variants can trivially be generated by simply reordering the layout of the subroutines. As with register swapping, subroutine permutation is a relatively weak malware morphing strategy, particularly with respect to statistical-based detection.

More general transpositions can be used. For example, the instructions

1. `OPCODE [R1] [R2]`
2. `OPCODE [R3] [R4]`

can be swapped, since they are independent of each other. Of course, such transposition can also be applied to group of instructions. Since the order of execution differs, transposition can be an effective means to evade signature detection.

2.2.3 Dead Code Insertion

In its simplest form, dead code is inserted into a program, but not executed. Alternatively, dead code can be executed, provided that it has no effect on the overall program function. Although more difficult, this latter approach can be more effective, since the dead code may be much more difficult to detect.

Dead code can be a highly effective means for evading malware detection, particularly with respect to statistical-based techniques. The dead code can be selected to mask the statistical properties of the underlying code. However, dead code insertion can be challenging at the assembly code level, since care must be taken so that addresses remain valid.

2.2.4 Instruction Substitution

An instruction (or group of instructions) can be substituted for another instruction (or group of instructions) with the same functionality. For example, `MOV R1, R2` can be replaced by `PUSH R1` followed by `POP R2`. As another trivial example, `XOR R1, R1` and `SUB R1, R1` both zero the contents of register R1. Instruction substitution is a powerful technique for evading signature detection and altering code statistics. However, instruction substitution is relatively difficult to implement at the assembly code level.

2.2.5 Formal Grammar Mutation

Formal grammar mutation is a formalization of existing morphing techniques [4, 10, 39]. Morphing engines can be viewed as non-deterministic automata, since transitions are possible from every symbol (i.e., instruction) to every other symbol [39]. By formalizing mutation techniques, we can apply formal grammar rules to create copies with wide variation. Figure 1 shows a simple polymorphic decryptor template and two

possible mutations of the decryptor achieved using the formal grammar in Figure 2. With this decryptor template and formal grammar combination, it is possible to generate 960 distinct decryptors [39].

2.3 LLVM

LLVM¹ [21] is a compiler infrastructure that has several novel features. LLVM supports a language independent instruction set where each instruction is a static single assignment (SSA), which means that each variable is assigned once and then cannot be reassigned [14, 19]. Static compilation is supported via late compilation of intermediate representation (IR) bytecode, analogous to the just-in-time (JIT) compiler in Java. The LLVM infrastructure is part of “The Lifelong Code Optimization Project” (LCO-Project) [16].

Most traditional static compilers (e.g., GCC) use three phases, and LLVM follows this approach. These three phases are a frontend, an optimizer, and a backend. Figure 3 illustrates the typical design of a three phase compiler.

The key function of the frontend is to parse the source code, check for syntax errors, and build a language-specific abstract syntax tree (AST). Using the AST, the optimizer manipulates instructions so as to optimize the code. For example, an optimizer removes duplicate code and redundant computations.

The compiler backend generates the machine-dependent representation of the code. Backend operations include instruction selection, register allocation, and instruction scheduling [20].

The key feature of the LLVM three-phase compiler design is that it supports multiple frontends and multiple backends, which is greatly simplified by its use of a common intermediate code representation. A frontend can be written for any language. The frontend converts the source code to LLVM IR bytecode which is machine and language independent. A backend can be written for any target platform by generating native code from this common intermediate representation [15, 20]. Figure 4 illustrates the LLVM compiler design.

The use of IR bytecode in LLVM effectively separates the frontend and backend components from each other. In addition, the use of IR bytecode supports lightweight runtime optimizations, cross-function or inter-procedural optimizations, program analysis, and aggressive restructuring transformations.

Figure 5 illustrates the structure of LLVM IR bytecode. The following sections are supported [27]:

1. Module — a container that holds functions and global variables
2. Functions — named, callable units of instructions
3. Global variables — variables that can be accessed by any function

¹“LLVM” was initially derived as an acronym for Low Level Virtual Machine. However, LLVM is now the official name—it is no longer an acronym.

Figure 6 shows a simple C function and its corresponding IR representation [22, 23].

The program life cycle from source program to executable in LLVM compiler is illustrated in Figure 7.

In LLVM IR bytecode, the logic is represented in the form of functions, and each function consists of a set of basic blocks. Each basic block consists of a set of instructions and all instructions in a basic block are executed sequentially. A variety of tools are available in the LLVM infrastructure to manipulate IR bytecode.

2.4 Hidden Markov Models

In this paper, we use hidden Markov models (HMM) as a tool to measure the effectiveness of our morphing strategy. In this section, we provide a very brief introduction to HMMs; see [33] for additional details.

Hidden Markov models can be viewed as a machine learning technique. We can train an HMM to fit a given observation sequence. The resulting model can then be used to score an unknown sequence to measure its similarity to the training data.

As the name suggests, a hidden Markov model includes a “hidden” Markov chain. Although this Markov chain is not directly observable, it is probabilistically related to a sequence of observed symbols. Figure 8 provides a generic illustration of an HMM, where the $\mathcal{O}_i$ are the observations, the matrix A drives the hidden Markov process, and the matrix B contains probability distributions that relate the hidden states to the observations.

Let π be the initial state probability distribution of the underlying Markov process. Then we denote an HMM as $\lambda = (A, B, \pi)$. The utility of HMMs derives largely from the fact that there are efficient algorithms to solve each of the following three problems [33].

- Problem 1: Given a model $\lambda = (A, B, \pi)$ and an observation sequence $\mathcal{O}$, we can compute $P(\mathcal{O} | \lambda)$. That is, we can score a sequence against a model.
- Problem 2: Given a model $\lambda = (A, B, \pi)$, we can determine an optimal state sequence for the Markov process. That is, we can “uncover” the hidden state sequence.
- Problem 3: Given an observation sequence $\mathcal{O}$, we can determine the model $\lambda = (A, B, \pi)$ that maximizes $P(\mathcal{O})$. That is, we can train a model to fit a given sequence of observations.

In this paper, we first train a model (Problem 3) on opcode sequences derived from a base piece of software. Then we use the trained model to score (Problem 1) morphed versions of this base software. Previous research has shown that HMMs are effective at detecting most metamorphic malware, and that HMMs can also be used to detect certain types of software piracy [13]. That is, HMMs have proven useful at detecting morphed or disguised versions of code. Consequently, HMM analysis provides a challenging test for any code morphing technique.

3 Design and Implementation

We have implemented two elementary metamorphic techniques at the LLVM IR bytecode level. Specifically, we use dead code insertion and function permutation. This morphing is available as an LLVM compile-time option.

Code morphing at the IR level offers the following advantages.

- A wide variety of front ends are available in LLVM. The supported languages include Objective-C, FORTRAN, Ada, Haskell, Java bytecode, Python, Ruby, Action Script, GLSL, D, and Rust. Using our tool, code written in any of these language can be morphed.
- The IR form is platform independent.
- At the IR level, virtual addresses are not assigned—addresses are first assigned at the bitcode (i.e., binary executable) level. Therefore, by morphing at the IR level, we avoid one of the major difficulties associated with morphing at the assembly level, namely, dealing with addresses.

Morphed copies of a program must have the same functionality as the base code. In addition, the higher the percentage of inserted or modified code, the more the morphed files should differ (on average) from the base file. In this research, we employ HMM analysis to measure the differences between files. As previously mentioned, HMMs have a proven record of being able to effectively “see through” metamorphic code. Consequently, if we can morph code sufficiently to defeat HMM-based analysis this will provide a strong indication of the success of our morphing strategy.

3.1 Morphing Technique

As we are morphing at IR bytecode level, it is difficult to adopt some of the techniques described in Section 2.2. For example, register swapping is relatively difficult to implement at the IR level. Therefore, to provide a proof of concept, we have restricted our code morphing to a combination of dead code insertion and subroutine permutation. We accomplish both of these morphing strategies by inserting randomly selected complete subroutines of dead code selected from other program files. In addition, the order of these dead subroutines is randomized. In this way, we create a significant amount of transposition and code variation between different morphed copies. In addition, we insert call statements to all dead code subroutines so that they are not trivially identifiable as dead code.

We have used `core-util` [18] Linux command files as the source of our dead code subroutines. These files include system level code to do operations that we would expect to be somewhat similar to our selected base code. By selecting morphing code that is similar to our base file, we are creating a more challenging task for our morphing engine, since the goal is to make the morphed code as different as possible from the base code.

The high-level architecture of our morphing engine appears in Figure 9. Next, we provide a detailed description of each of the three main phases of our morphing engine.

3.1.1 Dead Code Insertion

A base file, a morphing file (i.e., a source of dead code), and a dead code percentage are specified. Based on the dead code percentage, we determine the total number of lines we want to insert into the base file. We then select complete functions from the morphing file so that the total size approximates the number of lines we want to insert into the base file. These subroutines are integrated into the base file at the linking stage. The details of this first phase of our code morphing technique are given below.

1. Compile selected morphing file using the `llvm-gcc` command to generate its IR bytecode.
2. From this IR bytecode, determine function dependencies.
3. For each function, calculate its number of lines.
4. Based on the total number of dead code lines, use a greedy strategy to determine a subset of functions which best approximates the number of lines to be inserted.
5. Copy selected functions to a temporary IR bytecode file.
6. Create bitcode files for the base code and temporary IR bytecode file.
7. Merge these two files (using `llvm-link`).
8. If there are any subroutine naming conflicts, replace each offending name in the temporary IR bytecode file with a random string.
9. Delete the temporary IR bytecode file.

3.1.2 Call Dead Functions

In this pass, we use the LLVM optimizer to insert a call instruction for each dead code subroutine. The optimizer takes a function name as input. It then finds the `main` function definition in the IR bytecode and inserts a call type of instruction after every load type of instruction. The current implementation does not support structure type of parameters.

For each dead code subroutine, we perform the following steps.

1. Find the “function” object of the `main`.
2. Iterate over instructions in the function object.
3. If an instruction is of type load then insert a call instruction. To insert call instruction for dead function, iterate over its parameters and for each parameter, allocate memory and initialize with a random value.
4. Finally, insert a call instruction.

3.1.3 Function Permutation

The third pass performs function permutation by simply reordering functions in the IR bytecode file. This pass is straightforward and we omit the details. Additional details on the entire process can be found in [35].

4 Experimental Results

In this section, we use the HMM technique developed in [38] to test the effectiveness of our LLVM-based metamorphic code generator. We add increasing percentages of dead code to find the threshold at which HMM detector starts to fail. We show that after adding about 20% (or more) dead code, our metamorphic code is not reliably distinguished using this HMM technique. These results indicate that our LLVM-based morphing strategy is more effective than the hacker-produced metamorphic malware generators considered in previous research [38], and is at least as effective as an experimental metamorphic malware generator that was designed specifically to evade HMM-based detection [32].

For the experiments given here, we use spike fuzzer [31] as our base software. Fuzzing is a process of sending malformed data to an application to generate failures or errors in the application [12]. This base code was morphed using our LLVM metamorphic generator and the morphed versions were then analyzed using HMM-based analysis. Spike fuzzer consists of about 6000 lines of assembly code.

For each experiment, we generate 50 morphed copies by inserting dead code from different morphing files. As previously mentioned, the morphing files are randomly selected from coreutil Linux commands files [6].

Once the morphed files are generated, we use an HMM scoring technique similar to that in [13]. Previous research has consistently shown that the number of hidden states in the HMM does not significantly impact the quality of the file classification. Consequently, we only consider HMMs with $N = 2$ hidden states.

First, we train an HMM to model the base file. To obtain sufficient observations for training, we generated 50 copies of the base file, each having a 5% rate of morphing. We then trained an HMM on these 50 morphed files. We refer to this model as the “base HMM.” As discussed in [13], the purpose of the slight morphing at this stage is simply to prevent the base HMM from overfitting the available data in the base file. Consequently, we use a minimal amount of morphing at this step.

Next, we use this trained HMM to score 50 morphing files. Specifically, we score the coreutil Linux commands files that we use as our source of morphing code in the experiments described below.

We then conducted experiments where we morph the base file at each of the following rates: 10%, 20%, 30%, and, finally, 50%. In each case, we generated 50 morphed versions of the base file, with each file morphed at the given rate. These morphed copies were then scored using the base HMM and these scores were compared to the scores obtained for the morphing files as mentioned in the previous paragraph.

As the morphing percentage increases, we expect the scores of the morphed files to converge towards the scores of the morphing files. Note that all scores are normalized to a per opcode basis so that file size does not affect the results.

Figure 10 (a) through (d) contain our score results for 10% 20%, 30%, and 50% morphing, respectively. From these results, we see that after inserting 20% dead code, the scores are starting to merge, which indicates that the morphed base files are difficult for the HMM to distinguish from the morphing files. This is precisely the effect that we hope to achieve through code morphing.

The results in Figure 10 are summarized in the form of ROC curves in Figure 11. These ROC curves plot the false positive rate versus the true positive rate as the threshold is varied throughout the score range.

The area under the ROC curve (AUC) is equal to the probability that a classifier ranks a randomly chosen positive instance higher than a randomly chosen negative one [5]. The AUC values for the ROC curves in Figure 11 are given in the Table 1. Note that an AUC of 1.0 indicates ideal separation (i.e., no false positives or false negatives), while an AUC of 0.5 indicates that the classifier yields results that are no better than flipping a coin. After inserting 20% dead code, our HMM classifier does poorly, and at higher morphing rates, the rate of classification failure increases dramatically. Again, these results show that our code morphing technique is highly effective, at least with respect to this HMM classifier.

Table 1: ROC AUC statistic

Dead code insertion %	AUC
10	1.0000
20	0.8708
30	0.7724
50	0.5924

5 Conclusion and Future Work

In this paper, we presented and analyzed a novel code morphing technique based on LLVM IR bytecode. Our approach makes strong code morphing available as a compile-time option, and requires no special effort on the part of the software developer. As far as the authors are aware, this is the first general purpose code morphing tool of its kind.

Our metamorphic generator uses dead code insertion and function permutation. The dead code is in the form of functions copied from other programs. These dead functions are called within the program, which makes their detection and removal more challenging.

We tested the effectiveness of our code morphing using an HMM technique that has proven successful in metamorphic malware detection and for detection of certain types of software piracy. We verified that our morphing technique is highly effective, in the sense that an HMM cannot effectively distinguish our morphed code from other code, even at relatively low morphing rates.

There are many possible improvements to the metamorphic generator presented here. The dead code insertion could be improved by removing the dependence on complete subroutines—it would be possible to do such insertion at the level of basic blocks. Other powerful morphing techniques, such as instruction substitution, could be included. It would also be interesting to employ formal grammar mutation as a framework for implementing the morphing. Additional user control of morphing (via compile-time flags) would be valuable. Finally, improvements in the LLVM infrastructure itself would serve to make our code morphing techniques more robust. For example, in our current implementation, tools available within the LLVM framework could be used to analyze the morphed bitcode. However, if the LLVM bitcode is converted to, say, a Windows PE file, then the tools within LLVM are no longer directly applicable to such analysis.

References

- [1] S. Attaluri, S. McGhee, and M. Stamp, Profile hidden markov models and metamorphic virus detection, *Journal in Computer Virology*, 5(2):151–169, May 2009
- [2] J. Aycock, *Computer Viruses and Malware*, Springer-Verlag, New York 2006
- [3] D. Baysa, R. M. Low, and M. Stamp, Structural entropy and metamorphic malware, to appear in *Journal of Computer Virology and Hacking Techniques*, 9(4):179–192, November 2013
- [4] P. Beaucamps, Advanced Metamorphic Techniques in Computer Viruses, *International Conference on Computer, Electrical, and Systems Science, and Engineering*, CESSE’07, Venice, Italy, 2007
- [5] A. P. Bradley, The use of the area under the ROC curve in the evaluation of machine learning algorithms, *Pattern Recognition*, 30:1145–1159, 1997
- [6] Computer virus creation kit
<http://www.informit.com/articles/article.aspx?p=366890&seqNum=6>
- [7] The Mental Driller, Metamorphism in practice or “How I made MetaPHOR and what I’ve learnt”, 2002
<http://download.adamas.ai/dlbase/Stuff/VX%20Heavens%20Library/vmd01.html>
- [8] An example of metamorphic virus
<http://spth.virii.lu/main.html>
- [9] E. Filiol, *Computer Viruses: From Theory to Applications*, Volume 1, Birkhäuser, pp. 19–38, 2005

- [10] E. Filiol, Metamorphism, formal grammars and undecidable code mutation, *International Journal of Computer Science*, 2:70–75, 2007
- [11] X. Gao and M. Stamp, Metamorphic software for buffer overflow mitigation, *Proceedings of 3rd Conference on Computer Science and its Applications*, P. P. Dey and M. N. Amin, editors, San Diego, California, June 30, 2005
- [12] Introduction to fuzzing using spike fuzzer
<http://resources.infosecinstitute.com/intro-to-fuzzing/>
- [13] S. Kazi and M. Stamp, Hidden Markov models for software piracy detection, *Information Security Journal: A Global Perspective*, 22(3):140–149, 2013
- [14] C. Lattner and V. Adve, Architecture for a next generation GCC, *First GCC Annual Developer’s Summit*, May 2003
<http://llvm.org/pubs/2003-05-01-GCCSummit2003pres.pdf>
- [15] C. Lattner and V. Adve, A compilation framework for lifelong program analysis and transformation, *Proceedings of the 2004 International Symposium on Code Generation and Optimization*, 2004
http://www.cgo.org/cgo2004/papers/06_76_lattner_c.pdf
- [16] The Lifelong Code Optimization Project
<http://www-faculty.cs.uiuc.edu/~vadve/lcoproject.html>
- [17] D. Lin and M. Stamp, Hunting for undetectable metamorphic viruses, *Journal in Computer Virology*, 7(3):201–214, August 2011
- [18] Linux coreutils source code
<http://ftp.gnu.org/gnu/coreutil>
- [19] LLVM Programming Manual
<http://llvm.org/docs/ProgrammersManual.html>
- [20] LLVM Architecture
<http://www.aosabook.org/en/llvm.html>
- [21] The LLVM Compiler Infrastructure Project
<http://llvm.org/>
- [22] LLVM Helloworld in C
<http://projects.prabir.me/compiler/wiki/LLVMHelloworldInC.ashx>
- [23] LLVM IR Bytecode Format
<http://llvm.org/releases/1.3/docs/BytecodeFormat.html>
- [24] W. Ma, et al, Shadow Attacks: Automatically evading system-call behavior, *Journal in Computer Virology*, 8(1-2):1–13, May 2012
- [25] Open Malware, <http://www.offensivecomputing.net/>
- [26] Panda Security, Virus, worms, trojans and backdoors: Other harmful relatives of viruses, 2011
<http://www.pandasecurity.com/homeusers-cms3/security-info/about-malware/generalconcepts/concept-2.html>

- [27] J. Praher, A change framework based on the Low Level Virtual Machine Compiler Infrastructure, Thesis Report, Johannes Kepler University, April 2007
<http://llvm.cs.uiuc.edu/pubs/2007-04-PraherMSThesis.pdf>
- [28] N. Runwal, R. M. Low, and M. Stamp, Opcode graph similarity and metamorphic detection, *Journal in Computer Virology*, 8(1-2): 37–52, May 2012
- [29] G. Shanmugam, R. M. Low, and M. Stamp, Simple substitution distance and metamorphic detection, *Journal of Computer Virology and Hacking Techniques*, 9(3):159–170, August 2013
- [30] M. Sharif, et al, Impending malware analysis using conditional code obfuscation, College of Computing, Georgia Institute of Technology
<http://cyber4.us/sites/default/files/Impeding%20Malware%20Analysis%20Using%20Conditional%20Code%20Obfuscation-NDSS2008.pdf>
- [31] Spike Fuzzer Source Code
<http://www.immunitysec.com/resources-freesoftware.shtml>
- [32] S. Sridhara and M. Stamp, Metamorphic worm that carries its own morphing engine, *Journal of Computer Virology and Hacking Techniques*, 9(2): 49–58, May 2013
- [33] M. Stamp, A revealing introduction to hidden Markov models, 2012
<http://www.cs.sjsu.edu/~stamp/RUA/HMM.pdf>
- [34] M. Stamp, Risks of monoculture, Inside Risks 165, *Communications of the ACM*, 47(3):120, March 2004
<http://www.csl.sri.com/users/neumann/insiderisks04.html#165>
- [35] T. Tamboli, Metamorphic code generation from LLVM IR bytecode, Master’s Project 301, 2013
http://scholarworks.sjsu.edu/etd_projects/301/
- [36] A. H. Toderici and M. Stamp, Chi-squared distance and metamorphic virus detection, *Journal of Computer Virology and Hacking Techniques*, 9(1):1–14, February 2013
- [37] Virus Construction Kits
<http://computervirus.uw.hu/ch07lev1sec7.html>
- [38] W. Wong and M. Stamp, Hunting for metamorphic engines, *Journal in Computer Virology*, 2(3):211–229, December 2006
- [39] P. Zbitskiy, Code mutation techniques by means of formal grammars and automata, *Journal in Computer Virology*, 5(3):199–207, August 2009

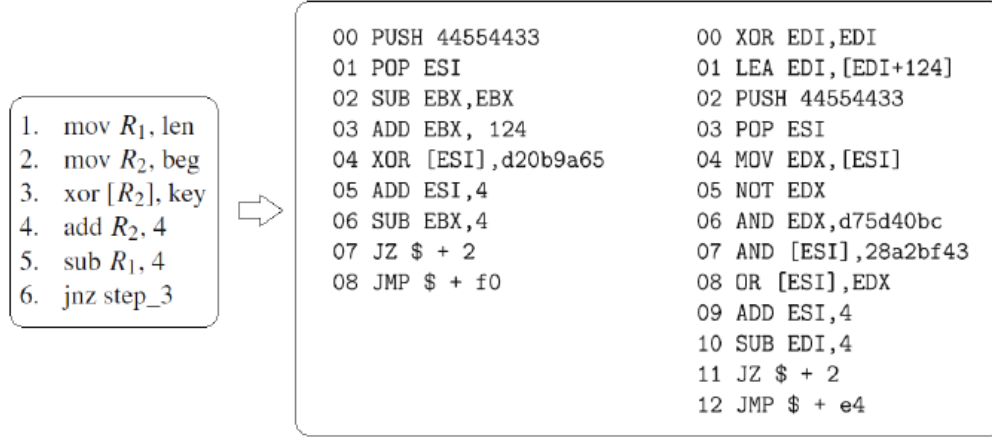


Figure 1: A simple polymorphic decryptor and two variants [39]

$A \rightarrow XB$
 $B \rightarrow Y_4 \varepsilon$
 $X \rightarrow X_1 X_2 | X_2 X_1$
 $X_1 \rightarrow GX_1 | \text{mov } R_1, \text{len} | \text{push len} \oplus \text{pop } R_1 | \text{xor } R_1, R_1 \oplus \text{lea } R_1, [R_1 + \text{len}] | \text{sub } R_1, R_1 \oplus \text{add } R_1, \text{len}$
 $X_2 \rightarrow GX_2 | \text{mov } R_2, \text{beg} | \text{push beg} \oplus \text{pop } R_2 | \text{xor } R_2, R_2 \oplus \text{lea } R_2, [R_2 + \text{beg}] | \text{sub } R_2, R_2 \oplus \text{add } R_2, \text{beg}$
 $Y_4 \rightarrow GY_4 | W_1 | S_4 W_4$
 $W_1 \rightarrow GW_1 | \text{xor } [R_2], \text{key } H_1$
 $W_1 \rightarrow \text{not } [R_2] \oplus \text{xor } [R_2], \text{key} \oplus \text{not } [R_2] H_1$
 $W_1 \rightarrow \text{mov } R_3, [R_2] \oplus \text{not } R_3 \oplus \text{and } R_3, \text{key} \oplus \text{and } [R_2], \neg \text{key} \oplus \text{or } [R_2], R_3 H_1$
 $H_1 \rightarrow GH_1 | \text{add } R_2, 4 H_2 | \text{sub } R_2, -4 H_2$
 $S_4 \rightarrow GS_1 | \text{sub } R_2, 4 | \text{add } R_2, -4$
 $W_2 \rightarrow GW_2 | \text{xor } [R_1][R_2], \text{key } H_2$
 $W_2 \rightarrow \text{not } [R_1][R_2] \oplus \text{xor } [R_1][R_2], \text{key} \oplus \text{not } [R_1][R_2] H_2$
 $W_2 \rightarrow \text{mov } R_3, [R_1][R_2] \oplus \text{not } R_3 \oplus \text{and } R_3, \text{key} \oplus \text{and } [R_1][R_2], \neg \text{key} \oplus \text{or } [R_1][R_2], R_3 H_2$
 $H_2 \rightarrow GH_2 | \text{sub } R_1, 4 \oplus \text{jnz } xxx | \text{sub } R_1, 4 \oplus \text{jz } yyy \oplus \text{jmp } xxx$
 $H_2 \rightarrow \text{add } R_1, -4 \oplus \text{jnz } xxx | \text{add } R_1, -4 \oplus \text{jz } yyy \oplus \text{jmp } xxx$
 $H_2 \rightarrow \text{sub ecx}, 3 \oplus \text{loop } xxx \Leftrightarrow R_1 \equiv \text{ecx}$

Figure 2: Formal grammar for decryptor mutation [39]

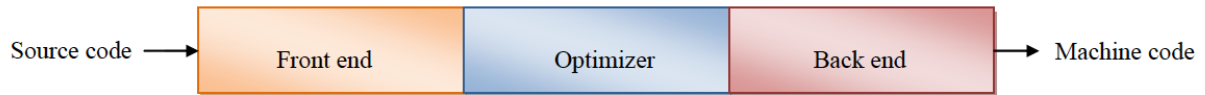


Figure 3: Three-phase compiler

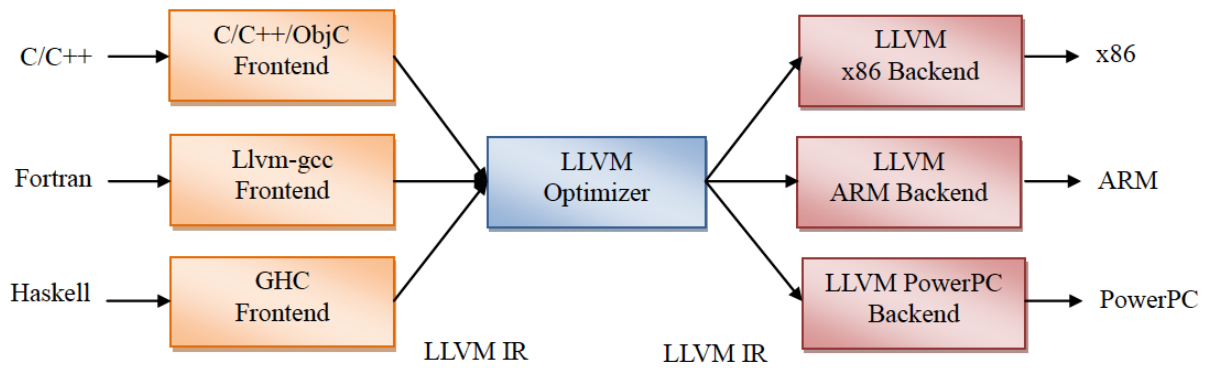


Figure 4: LLVM design [20]

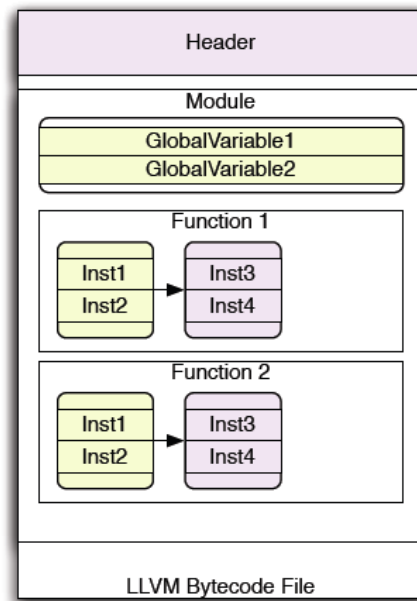


Figure 5: LLVM bytecode file format [27]

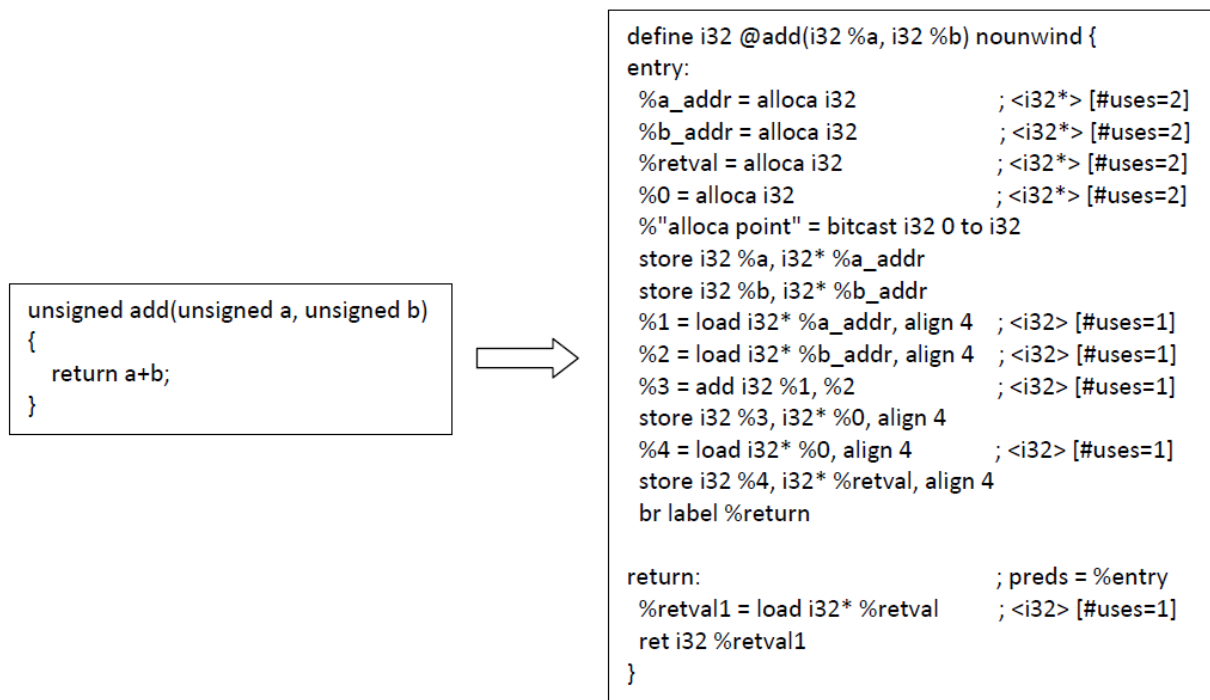


Figure 6: C code and corresponding IR bytecode

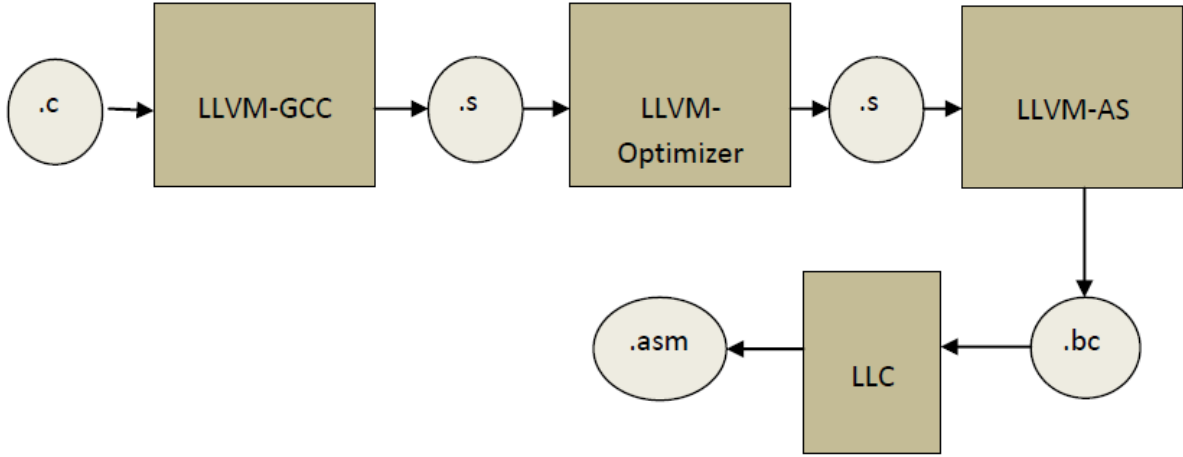


Figure 7: Program life cycle in LLVM compiler

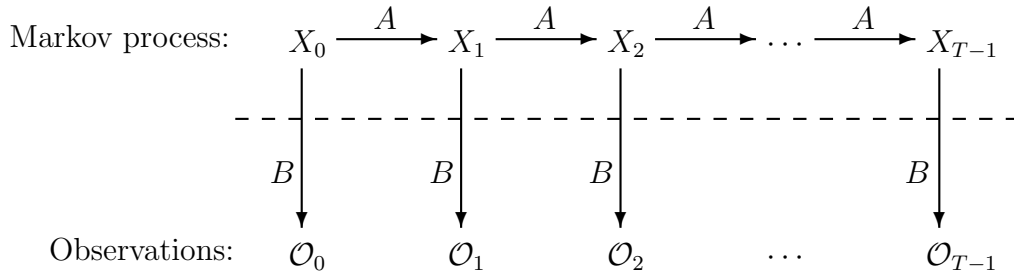


Figure 8: Generic HMM [33]

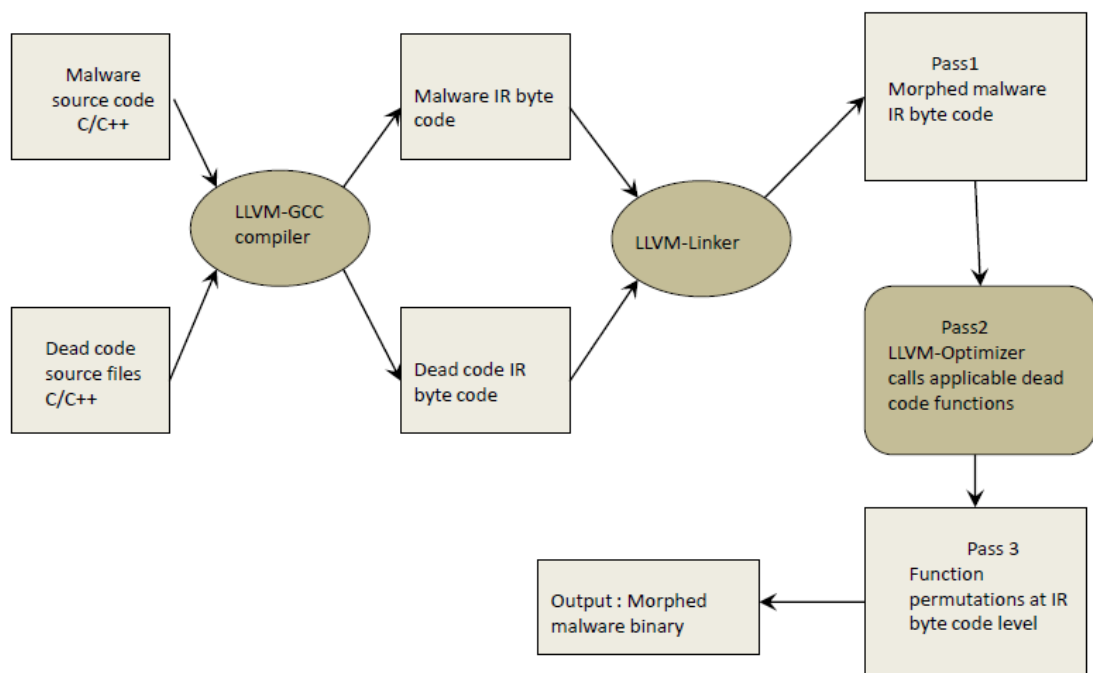
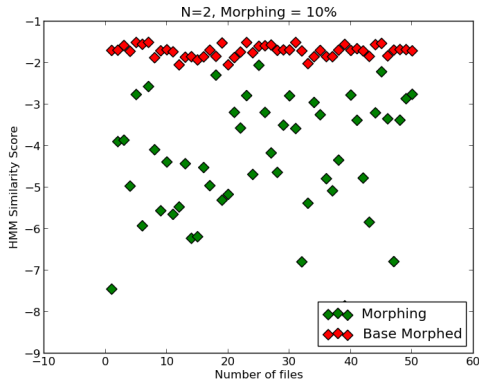
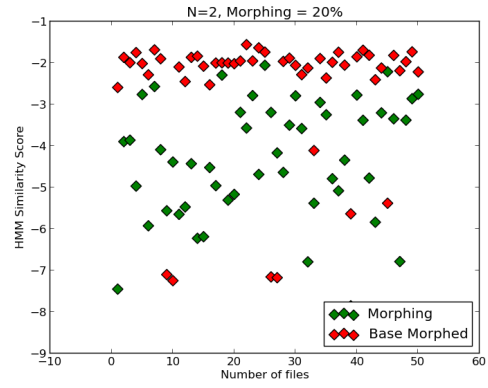


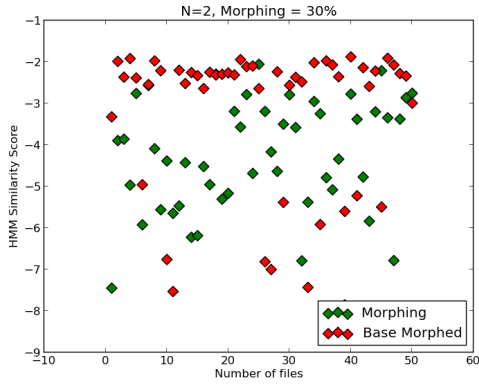
Figure 9: Metamorphic code generator architecture diagram



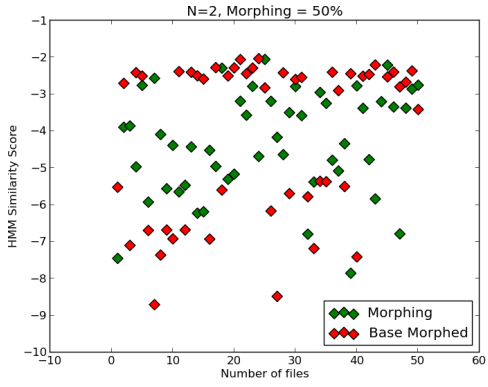
(a) 10% morphing



(b) 20% morphing

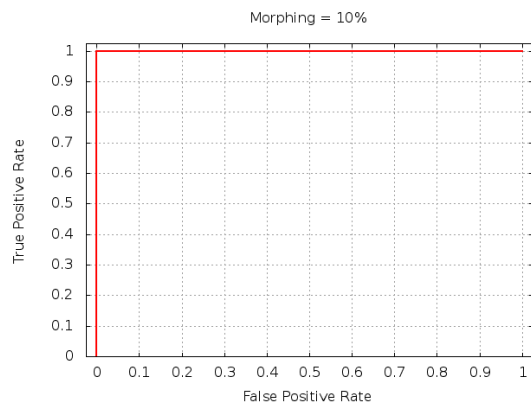


(c) 30% morphing

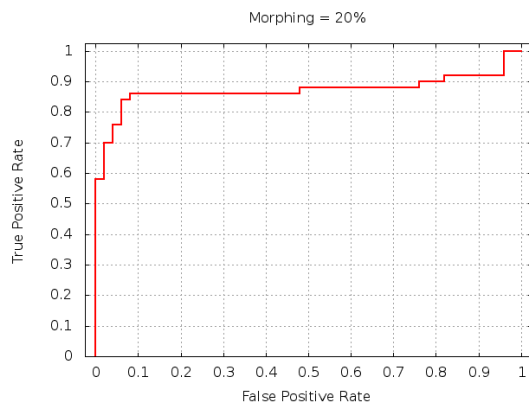


(d) 50% morphing

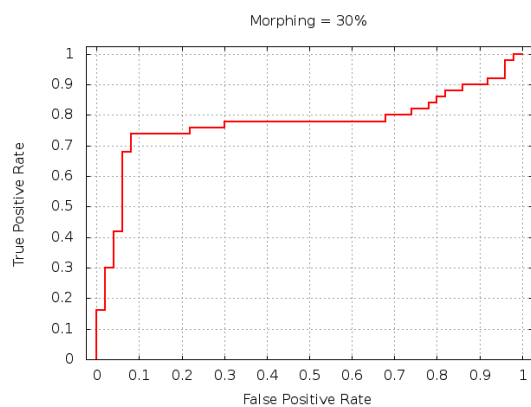
Figure 10: HMM scores for various morphing percentages



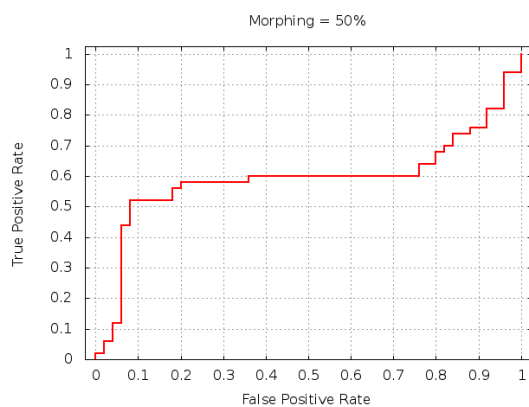
(a) 10% morphing



(b) 20% morphing



(c) 30% morphing



(d) 50% morphing

Figure 11: ROC curves for various morphing percentages